

Masterclass in C++

Additional Exercises to Exercise Set 1

Topic 1: Basic Class Creation (Based on Exercise 1)

Exercise 1.1

Create a class `Book` with private data members `title (string)`, `author (string)`, and `pages (int)`. Include public member functions to set and get each of these values separately.

Exercise 1.2

Design a class `Temperature` with a private data member `celsius`. Create public member functions `setCelsius()`, `getCelsius()`, `getFahrenheit()`, and `getKelvin()` that perform appropriate conversions.

Exercise 1.3

Implement a class `Time` with private data members `hours`, `minutes`, and `seconds`. Include public setter and getter functions for each member with basic validation (e.g., minutes and seconds should be 0-59).

Topic 2: Default Constructor (Based on Exercise 2)

Exercise 2.1

Write a class `Product` with private members `name (string)`, `price (double)`, and `quantity (int)`. Create a default constructor that initializes `name` to "Unnamed", `price` to 0.0, and `quantity` to 0.

Exercise 2.2

Create a class `Point2D` with private members `x` and `y` (both double). Implement a default constructor that initializes both coordinates to 0.0. Add a display function to show the point as (x, y).

Exercise 2.3

Design a class `Vehicle` with private members `brand (string)`, `model (string)`, and `year (int)`. Create a default constructor that initializes `brand` to "Unknown", `model` to "Unknown", and `year` to the current year (2024).

Topic 3: Parameterized Constructor (Based on Exercise 3)

Exercise 3.1

Modify the Product class to include a parameterized constructor that accepts name, price, and quantity as parameters. Ensure the constructor validates that price and quantity are non-negative.

Exercise 3.2

Update the Point2D class to include a parameterized constructor accepting x and y coordinates. Add another constructor that accepts a single value and uses it for both x and y.

Exercise 3.3

Enhance the Vehicle class with a parameterized constructor that accepts brand, model, and year. Validate that the year is between 1900 and the current year.

Topic 4: Access Specifier Practice (Based on Exercise 4)

Exercise 4.1

Create a class EmailAccount with private members email and password. Include public functions setEmail(), getEmail(), and changePassword() (which requires the old password). The password should never be retrievable.

Exercise 4.2

Design a class GradeBook with private array grades[10] and private member count. Create public functions addGrade(), getAverage(), and getHighestGrade(). Ensure direct access to the grades array is not possible.

Exercise 4.3

Implement a class Counter with a private static member totalCount and private member instanceCount. Provide public functions increment(), getCount(), and a static function getTotalCount().

Topic 5: Constructor Overloading (Based on Exercise 5)

Exercise 5.1

Create a class Box with private members length, width, and height. Implement four constructors:

- Default constructor (all dimensions = 1)
- Single parameter constructor (creates a cube)
- Three parameter constructor
- Copy constructor

Exercise 5.2

Design a class String with a private character pointer. Implement:

- Default constructor (empty string)
- Parameterized constructor accepting C-string
- Parameterized constructor accepting a single character
- Copy constructor (deep copy)

Exercise 5.3

Create a class Matrix2x2 with a private 2D array. Implement constructors:

- Default constructor (identity matrix)
- Single parameter constructor (scalar matrix)
- Four parameter constructor (all elements)
- Copy constructor

Topic 6: Initialization List (Based on Exercise 6)

Exercise 6.1

Create a class Account with const member accountNumber, reference member branch, and regular members holderName and balance. Use member initialization list to properly initialize all members.

Exercise 6.2

Design a class Node for a linked list with const member nodeID, reference member data, and pointer member next. Demonstrate proper initialization of all members using initialization list.

Exercise 6.3

Implement a class Configuration with multiple const members: maxUsers, timeout, and version, plus a reference member configFile. Show how to initialize these using initialization list with different constructors.

Topic 7: Multiple Constructors with Validation (Based on Exercise 7)

Exercise 7.1

Design a class PhoneNumber with private members for country code, area code, and number. Implement:

- Default constructor (sets to a valid default number)
- Parameterized constructor with validation (valid country codes, area codes)
- Copy constructor

Include format validation and a display function.

Exercise 7.2

Create a class IPv4Address with four octets. Implement:

- Default constructor (127.0.0.1)
- Parameterized constructor with validation (each octet 0-255)
- String constructor that parses "192.168.1.1" format
- Copy constructor

Exercise 7.3

Design a class CreditCard with card number, expiry date, and CVV. Implement multiple constructors with validation for:

- Valid card number length (16 digits)
- Future expiry date
- 3-digit CVV

Include appropriate error handling.

Topic 8: Private Constructor Pattern (Based on Exercise 8)

Exercise 8.1

Create a class Logger that implements singleton pattern with a private constructor. Include methods getInstance(), log(string message), and maintain a single log file throughout the application.

Exercise 8.2

Design a class ConfigurationManager with private constructor that loads settings from a file. Ensure only one instance exists and provide methods to read/update configuration values.

Exercise 8.3

Implement a class ResourcePool with private constructor that manages a limited number of resources (e.g., database connections). Use static method to control access and limit the number of instances to a fixed pool size.

Topic 9: Complex Constructor Chain (Based on Exercise 9)

Exercise 9.1

Create a hierarchy: Vehicle (base) → Car (derived) → ElectricCar (derived).

- Vehicle: engineNumber (const), manufacturer, year
- Car: doors, transmission type, fuel type
- ElectricCar: battery capacity, charging time, range
- Implement all constructor types with proper chaining
- Use static member for vehicle count

Exercise 9.2

Design hierarchy: Account → SavingsAccount → FixedDepositAccount.

- Account: accountID (const), holder name, balance
- SavingsAccount: interest rate, minimum balance
- FixedDepositAccount: tenure, maturity amount, penalty rate
- Demonstrate constructor delegation and initialization lists

Exercise 9.3

Implement: Shape → Shape2D → Rectangle → Square.

- Shape: shapeID (const), color
- Shape2D: x, y coordinates
- Rectangle: width, height
- Square: side length
- Show how Square constructor delegates to Rectangle
- Include static shape counter

Topic 10: Resource Management with RAII (Based on Exercise 10)

Exercise 10.1

Create a class FileHandler that:

- Opens a file in constructor (FILE* as private member)
- Implements copy constructor with proper handling
- Implements destructor to close file
- Provides read(), write(), and seek() methods

- Handles errors appropriately
- Bonus: Implement move semantics

Exercise 10.2

Design a class `NetworkConnection` that:

- Establishes socket connection in constructor
- Manages socket descriptor (private member)
- Implements deep copy constructor
- Closes connection in destructor
- Provides `send()` and `receive()` methods
- Implements timeout handling
- Bonus: Add SSL/TLS support management

Exercise 10.3

Implement a class `ThreadPool` that:

- Creates worker threads in the constructor
- Manages thread handles (private vector)
- Implements proper copy constructor (or delete it)
- Joins all threads in the destructor
- Provides task submission methods
- Handles thread synchronisation
- Bonus: Implement work-stealing queue